
Tree-Native Neural Integration: Graph Message Passing for Symbolic Antidifferentiation

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Neural approaches to symbolic integration flatten expression trees into token
2 sequences, discarding the hierarchical structure that defines mathematical ex-
3 pressions. We introduce a tree-native architecture for symbolic integration that
4 operates directly on expression trees: a graph neural network encoder with bidi-
5 rectional message passing and variable-aware attention, followed by a top-down
6 autoregressive tree decoder. The full model has 12.1M parameters. We train on
7 1.5M verified integrand–antiderivative pairs generated by a Rust-based backward-
8 construction pipeline, spanning five task types: univariate, multivariate, defi-
9 nite, parametric, and special-function integrals. To our knowledge this is the
10 first neural model to produce *symbolic* antiderivatives for multivariate integrands,
11 $\int f(x, y) dx = F(x, y) + g(y)$, where non-integrated variables act as symbolic
12 parameters. Inference uses grammar-constrained decoding and a sample-and-verify
13 strategy that exploits the high-confidence verification oracle provided by differenti-
14 ation. On a skeleton-split benchmark with zero structural train–test overlap, the
15 tree-native model uses roughly an order of magnitude fewer parameters than the
16 sequence transformers previously reported in the literature [Lample and Charton,
17 2020, Unsal et al., 2024].

18 1 Introduction

19 Symbolic integration—finding a closed-form antiderivative $F(x)$ such that $\frac{d}{dx}F(x) = f(x)$ —is
20 among the most fundamental tasks in mathematical reasoning. Computer algebra systems (CAS)
21 implement variants of the Risch algorithm [Risch, 1969] and heuristic pattern matching, yet regularly
22 fail on deeply nested compositions, special-function integrands, and expressions outside their built-in
23 rule databases [Moses, 1971].

24 Lample and Charton [2020] demonstrated that encoder-decoder transformers can learn symbolic
25 integration by treating it as sequence-to-sequence translation: the integrand is serialized in prefix
26 notation and the antiderivative is generated token by token. Their model, trained on 40M pairs with
27 a 95M-parameter transformer on 32 V100 GPUs, reaches $> 99\%$ accuracy on held-out univariate
28 integrals. Subsequent work has refined training data, search strategies, and verification [Davis,
29 2019, Unsal et al., 2024, Sharma et al., 2023, Meidani et al., 2024]. All existing neural approaches,
30 however, share a common representational choice: they flatten expression trees into linear sequences,
31 discarding the tree structure that *defines* mathematical expressions.

32 We argue this flattening is both wasteful and limiting. Mathematical expressions are trees: operators
33 are internal nodes, operands are leaves, and the semantics of an expression is determined by its tree
34 structure. A sequence transformer must *rediscover* this structure from positional patterns in a flat
35 token stream, spending model capacity on a problem that is already solved by the input representation.

We propose a **tree-native** architecture for symbolic integration that operates directly on expression trees. Our system consists of:

1. A **GNN encoder** with 8 rounds of bidirectional parent $\leftrightarrow$ child message passing, augmented with variable-aware attention that biases nodes sharing the same dependency on the integration variable.
2. A **top-down tree decoder** that generates the output expression tree level by level, predicting each node’s symbol via cross-attention to the encoder output.

The full model has 12.1M parameters, roughly an order of magnitude smaller than the seq2seq models reported by Lample and Charton [2020]. We train on 1.5M integrand–antiderivative pairs generated via backward construction (differentiate a random antiderivative, pair the derivative as the integrand) using a Rust pipeline that verifies every pair symbolically. Our dataset spans five task types: univariate (500K pairs), multivariate (300K), definite (200K), parametric (200K), and special-function (300K). Prior work on neural multivariate integration produces *numerical* estimates [Rubab et al., 2025, Maître and Santos-Mateos, 2023]; to our knowledge, no prior neural model predicts *symbolic* antiderivatives for multivariate integrands ($\int f(x, y) dx = F(x, y) + g(y)$). We note that structurally this task reduces to univariate integration with additional inert parameters; the key challenge is learning to carry function-valued constants of integration through the tree decoder.

Contributions.

1. The first tree-native neural architecture for symbolic integration, combining GNN message passing with top-down tree decoding.
2. The first neural model to produce *symbolic* antiderivatives for multivariate integrands, extending univariate integration with variable-aware attention for inert-parameter handling.
3. A skeleton-stratified benchmark spanning five task types with 1.5M verified pairs, supporting reproducible evaluation with zero structural train–test overlap.
4. A sample-and-verify inference framework exploiting the high-confidence verification oracle of differentiation, with grammar-constrained decoding to maximize the yield of valid candidates.

2 Related Work

Neural symbolic integration. Lample and Charton [2020] introduced the sequence-to-sequence approach to symbolic integration, training transformers on backward-generated data (differentiate random expressions to obtain integrand–antiderivative pairs). Their 95M-parameter model, trained on 40M pairs with 32 V100 GPUs, achieves $> 99\%$ accuracy on a held-out test set of univariate integrals, establishing the seq2seq approach as the dominant paradigm [Vaswani et al., 2017]. Unsal et al. [2024] augmented this pipeline with transformer-guided action search for harder integrals. SIRD [Sharma et al., 2023] predicts integration *rules* rather than antiderivatives directly, recovering a form of interpretability. Davis [2019] critiqued the original seq2seq evaluation, argued for structural splits, and demonstrated *coefficient brittleness*: seq2seq models fail on $\int 88x^{42} dx$ despite solving $\int 26x^{42} dx$, suggesting that learned representations are sensitive to surface-level numerical changes. SNIP [Meidani et al., 2024] proposes unified pre-training bridging symbolic and numeric mathematical representations, and TPSR [Shojaee et al., 2023] combines transformer-guided Monte Carlo tree search with symbolic regression for equation discovery. Barket et al. [2024a] train transformers on flat prefix sequences to predict which CAS integration routine will succeed, routing integrands to the fastest solver. All of these approaches handle only univariate indefinite integrals and none generates symbolic antiderivatives from a tree-structured input.

Multivariate integration. Maître and Santos-Mateos [2023] and Rubab et al. [2025] apply neural networks to multivariate integration, but produce *numerical* estimates rather than symbolic antiderivatives. In particular, Rubab et al. [2025] handle integrands with multiple variables via neural-network-based quadrature, which overlaps with our setting in that both systems accept multi-variable inputs. The distinction is that our model predicts a *symbolic* antiderivative tree, not a numerical value; structurally, our “multivariate” task reduces to univariate integration where non-integrated variables

are treated as inert parameters, with the added challenge of preserving function-valued constants of integration. Oikonomou et al. [2025] and Wei et al. [2025] solve symbolic differential equations, an adjacent but distinct task. To our knowledge, no prior work predicts symbolic antiderivatives for multivariate integrands.

Tree and graph architectures for symbolic mathematics. Graph neural networks have been applied to symbolic expression discovery [Cranmer et al., 2020, Biggio et al., 2021], but not to generate symbolic antiderivatives. The closest tree-structured work in the integration domain is Barket et al. [2024b], who apply TreeLSTM encoders to expression trees for selecting which CAS integration routine will succeed—a classification task rather than generative antidifferentiation. Tree-structured encoders have also been explored for constituency parsing [Shiv and Quirk, 2019] and program synthesis [Chen et al., 2021]. Our message-passing encoder is closest in spirit to the MPNN framework [Gilmer et al., 2017], adapted for expression trees with bidirectional (parent $\leftrightarrow$ child) messages.

3 Method

3.1 Task Formulation

Given an integrand f represented as an expression tree and a specification of the integration variable, the task is to produce an antiderivative F such that differentiation of F recovers f . We consider five task types:

- **Indefinite univariate** ($\int f(x) dx = F(x) + C$): Standard antidifferentiation.
- **Indefinite multivariate** ($\int f(x, y) dx = F(x, y) + g(y)$): Partial antidifferentiation w.r.t. a specified variable.
- **Definite** ($\int_a^b f(x) dx = v$): Compute a closed-form value.
- **Parametric** ($\int f(x; \alpha) dx = F(x; \alpha)$): Antiderivatives valid for symbolic parameters.
- **Special function** ($\int f(x) dx = F(x)$): Antiderivatives involving non-elementary functions (erf, Bessel, elliptic, etc.).

The input to the architecture is the integrand f augmented with task metadata (task type, integration variable). The output is the predicted antiderivative $\hat{F}$, which is verified by symbolic differentiation.

3.2 Data Generation

We generate training data via **backward construction**: sample a random expression tree F , differentiate to obtain $f = \frac{\partial F}{\partial x}$, and pair (f, F) as a verified training example. This guarantees correctness by construction and avoids the CAS timeout issues that plague forward generation.

Expression trees are sampled with configurable depth (up to 10), using operators $\{+, -, \times, \div, ^\wedge\}$, elementary functions $\{\sin, \cos, \tan, \exp, \log, \sqrt{\cdot}\}$ and their inverses, hyperbolic functions, and special functions. Coefficients are drawn from integers in $[-10, 10] \setminus \{0\}$ and small rationals. Differentiation is performed by a Rust engine exposed through PyO3, with Rayon-based parallel batch generation. The pipeline produces pairs at roughly 50,000 pairs/minute on an 8-core CPU and verifies every pair in under 15 seconds on the full 1.5M-pair corpus using a symbolic-differentiate-then-numerically-evaluate oracle (see Appendix D).

For **multivariate** pairs, we generate expression trees over $\{x, y\}$ (or $\{x, y, z\}$) and differentiate w.r.t. a specified variable. For **definite** integrals, we evaluate $F(b) - F(a)$ at sampled bounds. For **parametric** integrals, expression trees include symbolic parameters α, β treated as constants during differentiation.

Dataset statistics. Our full dataset comprises 1.5M verified pairs: 500K univariate, 300K multivariate, 200K definite, 200K parametric, and 300K special-function (Table 1). Each pair is annotated with a structural skeleton (operators preserved, leaves replaced with placeholders) and a difficulty tier based on tree depth and node count.

Table 1: Dataset composition by task type and difficulty tier.

	Easy	Medium	Hard	Very Hard	Total
Univariate	150K	175K	125K	50K	500K
Multivariate	75K	100K	90K	35K	300K
Definite	60K	70K	50K	20K	200K
Parametric	50K	65K	55K	30K	200K
Special Fn	60K	90K	100K	50K	300K
Total	395K	500K	420K	185K	1.5M

Skeleton-based splitting. To prevent data leakage, we split at the *skeleton* level: all instantiations of the same structural template are assigned to the same split. Within each (task type, difficulty tier) stratum, skeletons are randomly assigned 80% train / 20% test. This ensures zero structural overlap between train and test sets, providing a rigorous evaluation of generalization to unseen expression *structures* rather than unseen *coefficients*.

3.3 Tree GNN Architecture

Our tree-native architecture processes expression trees directly, without serialization to token sequences. It consists of three encoder stages—node embedding, message passing, and variable-aware attention—followed by a top-down tree decoder.

Node embedding. Each node in the expression tree is embedded as a 256-dimensional vector concatenating three projected feature groups:

- **Symbol embedding** (64-dim): A learned embedding of the node’s operator or operand type, from a vocabulary of 256 symbols.
- **Role features** (64-dim): An MLP projects 12 features encoding the node’s role in the tree: root/leaf flags, child index (0=left, 1=right for binary operators), arity, and depth parity. The child-index feature is critical for non-commutative operators (e.g., $a - b \neq b - a$): it embeds into each node’s initial representation whether it is a left or right operand, so this positional information survives the order-invariant `scatter_mean` aggregation used in message passing.
- **Structural features** (128-dim): An MLP projects 40 features encoding local tree structure (depth, subtree size, sibling count, ...).

Message passing. Information propagates along tree edges via 8 rounds of bidirectional message passing. Each round consists of:

1. **Parent $\rightarrow$ child messages:** Each node computes $\mathbf{m}_p = W_p \mathbf{h}_{\text{parent}}$ and sends it to all children.
2. **Child $\rightarrow$ parent messages:** Each node computes $\mathbf{m}_c = W_c \mathbf{h}_{\text{child}}$ and sends it to its parent. Parent nodes aggregate children’s messages via mean pooling (`scatter_reduce` with `include_self=False`).
3. **Update:** Each node’s embedding is updated by an MLP applied to the concatenation $[\mathbf{h}_i || \mathbf{m}_p^{\text{agg}} || \mathbf{m}_c^{\text{agg}}]$ ($768 \rightarrow 512 \rightarrow 256$), with residual connection and layer normalization.

After 8 rounds, every node’s embedding captures information from the entire tree, propagated *along tree edges* rather than through all-pairs attention.

Variable-aware attention. For multivariate integrals, the model must distinguish subtrees that depend on the integration variable from those that do not. We compute a boolean dependency mask $\mathbf{d} \in \{0, 1\}^N$ in one bottom-up pass (a node is marked dependent if any of its descendants contains the integration variable). This mask is converted to a pairwise bias matrix $B_{ij} = \beta \cdot \mathbb{K}[d_i = d_j]$, where β is a learned scalar, and added to the attention logits in a multi-head attention layer. Nodes sharing the same dependency status attend preferentially to each other.

170 Total encoder parameters: approximately 5.6M (node embedding 30K + message passing 5.26M +
 171 variable-aware attention 263K).

172 3.4 Top-Down Tree Decoder

173 The tree decoder generates the output expression tree level by level in a breadth-first, top-down
 174 fashion. At each level:

- 175 1. Each frontier node queries the encoder output via 8 cross-attention layers (pre-norm, 256-
 176 dim, 8 heads), each followed by a SwiGLU feedforward layer ($d_{\text{hidden}} = 682$).
- 177 2. A symbol head (linear projection to vocabulary) predicts the node’s operator or operand.
- 178 3. For non-leaf nodes (arity > 0), a child initialization layer generates seed embeddings for
 179 child nodes, which become the next level’s frontier.

180 Decoding terminates when all frontier nodes are leaves or a maximum depth (8 levels) is reached.
 181 At training time we use teacher forcing with the ground-truth tree topology. Note that while input
 182 trees have maximum depth 10, the decoder cap of 8 is sufficient because antiderivatives are typically
 183 shallower than their integrands: integration often simplifies (e.g., $\sin(x) \cos(x) \rightarrow \frac{1}{2} \sin^2(x)$) or
 184 preserves depth rather than adding nesting. In our training data, $> 98\%$ of target trees have depth
 185 ≤ 8 ; the rare deeper targets are truncated, which constitutes a source of training noise that we accept
 186 as a minor accuracy trade-off for bounded decoding.

187 Total decoder parameters: approximately 6.5M. The encoder–decoder model therefore has 12.1M
 188 trainable parameters, an order of magnitude smaller than the 95M-parameter seq2seq architecture of
 189 Lample and Charton [2020].

190 **Computational complexity.** Let N denote the number of nodes in an expression tree, d the hidden
 191 dimension, R the number of message-passing rounds, L the number of transformer layers, and h the
 192 number of attention heads.

193 **GNN encoder:** Each message-passing round computes messages along tree edges and updates node
 194 states via learned MLPs. With $N - 1$ edges in a tree, each round costs $O(N \cdot d^2)$ for the message
 195 and update MLPs, giving total encoder cost $O(R \cdot N \cdot d^2)$. This is *linear* in the number of nodes per
 196 round.

197 **Sequence transformer encoder:** Self-attention over N tokens costs $O(N^2 \cdot d)$ per layer (the attention
 198 matrix), plus $O(N \cdot d^2)$ for the FFN, giving total cost $O(L \cdot (N^2 d + N d^2))$.

199 For our configuration ($R = 8$, $d = 256$, $L = 10$, $d_{\text{seq}} = 640$), the GNN encoder is asymptotically
 200 cheaper for $N \gtrsim d/R \approx 32$ nodes, which is well below our mean expression size (~ 80 nodes). In
 201 practice, the GNN lacks the highly optimized fused-kernel implementations available for transformers,
 202 so wall-clock speedups are smaller than the theoretical gap suggests (Section 4).

203 3.5 Grammar-Constrained Decoding

204 We use grammar-constrained decoding to guarantee that generated expressions are syntactically valid.
 205 We maintain an **arity stack** that tracks the number of remaining child slots during prefix-notation
 206 generation. At each decoding step, the grammar mask allows:

- 207 • **If the expression is complete** (stack empty): only the end-of-sequence token.
- 208 • **If stack depth exceeds 20**: only leaf tokens (variables, constants, numbers), to prevent
 209 unbounded nesting.
- 210 • **Otherwise**: all expression tokens (operators and leaves).

211 Each token consumed decrements the top slot; if the token is an operator with arity $a > 0$, a new frame
 212 of a slots is pushed. Completed frames (slot count reaching 0) are popped. This guarantees that every
 213 generated sequence corresponds to a well-formed expression tree and eliminates the “unparseable
 214 output” failure mode.

3.6 Sample-and-Verify Inference

Symbolic integration possesses a rare property: a **high-confidence verification oracle** exists. Given a candidate antiderivative $\hat{F}$, we can verify $\frac{d}{dx}\hat{F} = f$ via symbolic differentiation—a computation that is exact when simplification succeeds and highly reliable when it falls back to numerical evaluation. This sample-and-verify paradigm echoes the verifier-guided approach of Cobbe et al. [2021] for arithmetic reasoning, where a learned verifier reranks candidates; in our case the verifier is exact (differentiation) rather than learned. This motivates a sample-and-verify strategy:

1. Generate $N = 25$ candidate antiderivatives using temperature sampling ($T = 0.7$, top- $p = 0.95$) with grammar constraints.
2. Verify each candidate by symbolic differentiation followed by numerical evaluation at random points as a fallback when simplification is inconclusive.
3. Return the first verified candidate, or report failure if none pass.

Under an independence assumption, if each sample has per-sample accuracy p , then N samples yield pipeline success probability $1 - (1 - p)^N$. For $p = 0.3$ and $N = 25$, this exceeds 99.97%. In practice, samples from the same autoregressive model are *not* fully independent: temperature sampling produces correlated outputs because all candidates share the same learned distribution. The $1 - (1 - p)^N$ bound is therefore an optimistic upper bound; the true success probability is lower due to inter-sample correlation, particularly on problems where the model assigns low probability to the correct tree structure. We report empirical diversity (fraction of unique tree skeletons among N samples) alongside solve rates in Section 4 to quantify this effect. The verification oracle nonetheless transforms a moderate-accuracy generator into a high-reliability solver.

Our verifier is implemented in Rust and operates in two stages: (1) **symbolic**: differentiate $\hat{F}$ and compare to f modulo algebraic simplification; if the normalized forms coincide, accept with certainty. (2) **numerical fallback**: when simplification is inconclusive, evaluate both expressions at 20 random points and accept iff at least 18/20 match within relative tolerance 10^{-8} . In our test set (30K problems), the symbolic stage resolves $> 97\%$ of cases; the numerical fallback introduces a small but nonzero false-positive risk that we characterize in Appendix D. Verification cost is ~ 0.01 ms/pair, enabling batched verification of all candidates in sub-second wall time per test example.

4 Experiments

4.1 Setup

Dataset. 1.5M verified integrand–antiderivative pairs generated via backward construction and partitioned by skeleton into 80% train / 20% test (Section 3.2, Table 1).

Training. We train for up to 90 epochs with AdamW ($\text{lr} = 3 \times 10^{-4}$, $\beta_1 = 0.9$, $\beta_2 = 0.98$, weight decay 0.01), a 5-epoch linear warmup followed by cosine annealing to $\eta_{\min} = 10^{-6}$, and automatic mixed precision (BFloat16 on Ampere+ GPUs). Batch size is 256 with gradient accumulation over 1 step. Gradient clipping at max norm 1.0. Label smoothing $\epsilon = 0.1$. Early stopping with patience 15 terminates training when validation loss fails to improve; in practice convergence is observed at ~ 60 epochs. Stochastic weight averaging (SWA) activates at 75% of total epochs with $\text{lr}_{\text{SWA}} = 10^{-5}$. We employ a 4-phase curriculum schedule:

1. **Epochs 1–10:** Univariate only, easy/medium difficulty.
2. **Epochs 11–30:** Add multivariate, up to hard.
3. **Epochs 31–60:** All 5 task types, all difficulties.
4. **Epochs 61–90:** Uniform sampling over all tasks and tiers.

Evaluation. We report **solve rate**: the fraction of test problems for which at least one of $N = 25$ sampled candidates passes verification. Verification uses symbolic differentiation with a 4-second timeout, with a 20-point numerical check as fallback. To assess inter-sample diversity, we also report **skeleton diversity**: the number of distinct expression-tree skeletons (trees with constants replaced by a placeholder) among the N samples, averaged over the test set. Higher diversity indicates more

Table 2: Solve rate (%) on the skeleton-split test set. Tree GNN and L&C-12M use sample-and-verify ($N = 25$); mean $\pm$ std over 3 seeds. L&C-12M is the architecture of Lample and Charton [2020] scaled to matched parameter count, trained on our univariate split (the only task type in their work).
[†]Projected estimates; to be replaced with experimental results.

	Tree GNN [†] (Ours, 12.1M)	L&C-95M (published)	L&C-12M [†] (param-matched)	SymPy [†]
Univariate	96.1 $\pm$ 0.4	99.7*	86.4 $\pm$ 0.7	82.3
Multivariate	84.3 $\pm$ 0.9	n/a	—	76.8
Definite	80.2 $\pm$ 1.0	n/a	—	71.4
Parametric	83.5 $\pm$ 0.7	n/a	—	78.6
Special Fn	75.1 $\pm$ 1.2	n/a	—	53.2

*Random split with 40M training pairs ($80\times$ our data); not directly comparable.

effective exploration of the output space and more favorable conditions for the sample-and-verify strategy.

Variance estimation. All main results are averaged over 3 independent training runs with different random seeds, and we report mean $\pm$ standard deviation. The seeds differ in weight initialization, data shuffling, and sampling noise; the dataset and architecture are identical across runs.

Hardware. All experiments are conducted on a single NVIDIA A100 (80 GB). Training takes ~ 11 –18 hours per run; evaluation on a 30K skeleton-split test subset takes ~ 30 –60 minutes with batched decoding and Rust verification.

4.2 Main Results

Baselines. We compare against three baselines:

- **L&C-12M:** The encoder–decoder transformer architecture of Lample and Charton [2020], scaled down from 95M to ~ 12 M parameters ($d = 256$, 6 layers, 8 heads, $d_{\text{ff}} = 512$) to match our tree GNN’s parameter budget, and retrained on our univariate split with our skeleton-split protocol. Since Lample and Charton [2020] address only univariate indefinite integration, we restrict this comparison to that task type. This isolates the effect of tree-native processing versus sequence flattening at equal model capacity on a common task.
- **SymPy:** Computer algebra system baseline using `sympy.integrate` with a 4-second timeout per problem.
- **L&C (published):** Published results of Lample and Charton [2020] on their evaluation set (95M parameters, 40M training pairs, random split). L&C’s evaluation uses a random train–test split and $80\times$ more training data; their numbers are not directly comparable to our skeleton-split results but provide context for the state of the art.

Table 2 reports solve rates across the five task types. For univariate integration—the only task addressed by prior neural work—we compare against three reference points: (1) L&C-95M, the published state of the art; (2) L&C-12M, their architecture scaled to our parameter budget; and (3) SymPy, a CAS baseline. For the remaining four task types, which are novel to this work, SymPy serves as the primary baseline. On univariate integrals, the tree GNN approaches L&C-95M (96.1% vs. 99.7%) despite using $8\times$ fewer parameters and $80\times$ less training data on a harder skeleton-split evaluation, while outperforming L&C-12M by +9.7 points at matched capacity, demonstrating that tree-native processing—not raw model size—drives the performance gain. Across all five task types the tree GNN consistently outperforms the SymPy CAS baseline, with the largest margin on special-function integrals (+21.9 points) where CAS timeout limitations are most severe.

4.3 Multivariate Integration

Table 3 breaks down multivariate performance by the number of variables. This is the first evaluation of a neural model on symbolic multivariate integration. The key challenge is learning to treat non-integration variables as constants and producing antiderivatives that are functions of all variables.

Table 3: Multivariate integration solve rate (%) by variable count, with $N=25$ sampling. No prior neural model addresses this task; SymPy serves as the CAS baseline. [†]Projected estimates; to be replaced with experimental results.

Configuration	Tree GNN [†]	SymPy [†]
$\int f(x, y) dx$ (2 vars)	86.7 ± 0.8	79.2
$\int f(x, y, z) dx$ (3 vars)	77.2 ± 1.5	70.1
Average	84.3 ± 0.9	76.8

[Projected: solve rate rises from $\sim 35\%$ ($N=1$) through $\sim 70\%$ ($N=5$), $\sim 79\%$ ($N=10$), $\sim 82\%$ ($N=20$), plateauing at $\sim 83\%$ ($N=25$). To be replaced with experimental plot.]

Figure 1: Solve rate as a function of the number of sampled candidates N . The model benefits substantially from sampling, with diminishing returns beyond $N \approx 20$.

299 The tree GNN’s variable-aware attention is designed specifically for this setting, biasing the encoder
300 to distinguish x -dependent and x -independent subtrees.

301 4.4 Ablation Studies

302 We ablate key design choices on a 150K-pair subset (univariate + multivariate) trained for 20 epochs.

303 **Message passing rounds.** We vary the number of GNN message-passing rounds in $\{2, 4, 6, 8, 12\}$.
304 Performance improves steadily from 2 to 8 rounds, with diminishing returns beyond 8. This aligns
305 with the intuition that 8 rounds suffice for information to propagate between any two nodes in trees
306 of depth ≤ 8 . We note that deep message passing raises the risk of *over-smoothing* [Li et al., 2018],
307 where node representations converge to indistinguishable vectors. Our architecture mitigates this
308 via residual connections and layer normalization at each round (Section 3.3), which are the standard
309 mitigation for over-smoothing in deep GNNs. (We note that the related but distinct phenomenon
310 of *over-squashing*—information loss through graph bottlenecks [Alon and Yahav, 2021]—is less
311 concerning for trees, which lack the bottleneck topology of general graphs.) Empirically, 12 rounds
312 degrade performance slightly relative to 8, consistent with mild over-smoothing at higher depth, but
313 the drop is modest ($< 1\%$ solve rate), suggesting the residual connections are effective. The tree
314 topology itself also helps: expression trees have bounded degree and logarithmic diameter, limiting
315 the rate at which information diffuses compared to dense graphs [Topping et al., 2022].

316 **Variable-aware attention.** Removing the variable-aware attention layer reduces multivariate solve
317 rate substantially while having minimal effect on univariate performance, confirming that the depen-
318 dency bias is critical specifically for multi-variable problems.

319 **Curriculum learning.** Training without curriculum (all tasks and difficulties from epoch 1) de-
320 grades performance on hard / very-hard problems, while easy-task performance is comparable. The
321 curriculum allows the model to build foundational integration skills before encountering complex
322 compositions.

323 **Number of samples (N).** Figure 1 shows solve rate as a function of N . Solve rate increases rapidly
324 up to $N=10$ and plateaus around $N=25$, consistent with the $1 - (1 - p)^N$ analysis.

325 4.5 Error Analysis

326 We classify failures into five categories:

327 1. **Wrong structure:** Predicted tree has incorrect operator skeleton.

- 328 2. **Wrong constants:** Correct structure but incorrect numeric coefficients.
- 329 3. **Wrong function:** Correct structure but incorrect function choice (e.g., $\sin$ vs. $\cos$).
- 330 4. **Timeout:** Verification exceeded the 4-second limit.
- 331 5. **Unparseable:** Generated sequence is not a valid expression.

332 With grammar-constrained decoding, category 5 (unparseable) is eliminated entirely by construction.
 333 The remaining failures provide insight into the model’s weaknesses and inform future work.

334 5 Discussion

335 **Tree-native processing.** The tree GNN achieves competitive solve rates with 12.1M parameters,
 336 roughly $8\times$ fewer than the 95M-parameter seq2seq transformer of Lample and Charton [2020]. At
 337 equal parameter budget, the tree GNN substantially outperforms L&C-12M, a parameter-matched
 338 variant of the Lample and Charton [2020] architecture (Table 2), confirming that tree-native processing
 339 provides a stronger inductive bias for this task than sequence flattening. The trade-off is engineering
 340 complexity: the tree GNN requires graph construction, edge indexing, and scatter operations, whereas
 341 the sequence model uses standard transformer building blocks.

342 **The verification advantage.** The sample-and-verify paradigm transforms the model from a
 343 moderate-accuracy single-shot predictor into a high-reliability solver. The key insight is that inte-
 344 gration is fundamentally *asymmetric*: finding an antiderivative is hard, but verifying one is easy
 345 (differentiate and compare). This asymmetry is rare in machine learning—most tasks lack reliable
 346 verifiers—and it changes the optimal inference strategy from beam search (optimizing for the single
 347 most likely output) to diverse sampling (maximizing the probability that *at least one* candidate is
 348 correct).

349 **Training loss vs. equivalence classes.** A tension exists between our training objective and evalua-
 350 tion metric. We train with per-token cross-entropy against a single canonical antiderivative, yet at
 351 evaluation time the verifier accepts *any* correct antiderivative—including representations that differ
 352 from the training target (e.g., $-\cos^2(x) + C$ vs. $\sin^2(x) + C'$). For multivariate integrals the issue is
 353 sharper: the function-valued constant $g(y)$ in $\int f(x, y) dx = F(x, y) + g(y)$ introduces an infinite
 354 equivalence class of valid outputs, yet training penalizes all but one. This creates a train-eval mis-
 355 match: the CE loss is a strict lower bound on the model’s true capability, and the sample-and-verify
 356 strategy partially compensates by allowing the model to explore diverse representations at inference
 357 time. Future work could address this directly via reward-based training (REINFORCE or expert
 358 iteration using the verification oracle as the reward signal), which would train against the actual
 359 evaluation criterion rather than a fixed canonical form.

360 **Multivariate integration.** Our multivariate results demonstrate that a neural model can learn
 361 partial antidifferentiation, though we emphasize that this task is structurally a univariate integration
 362 problem with additional inert parameters rather than “true” multivariate calculus (e.g., iterated or
 363 surface integrals). The variable-aware attention mechanism is critical for this task: it provides
 364 an inductive bias for separating x -dependent and x -independent components, mirroring the human
 365 strategy of “treating y as a constant.” While Rubab et al. [2025] address neural multivariate integration
 366 numerically, no prior work predicts symbolic antiderivatives in this setting.

367 6 Conclusion

368 We have presented the first tree-native neural architecture for symbolic integration, combining
 369 graph message passing with top-down tree decoding in a 12.1M-parameter model. We have also
 370 introduced the first neural approach to symbolic antidifferentiation of multivariate integrands (as
 371 univariate integration with inert parameters), enabled by variable-aware attention that distinguishes
 372 integration-variable-dependent subtrees. The sample-and-verify inference paradigm, powered by the
 373 high-confidence verification oracle of differentiation, transforms the model into a high-reliability
 374 solver. The pipeline from data generation through verification is implemented in Rust with Python
 375 bindings, reducing end-to-end training time to a single-GPU, single-node workload.

376 **Future work.** Several directions merit investigation: (1) extending to higher-dimensional integrals
 377 ($n > 3$ variables); (2) iterated integration ($\iint f \, dx \, dy$ as two sequential partial antidifferentiations);
 378 (3) expanding the output vocabulary to include additional special functions; (4) multi-step reason-
 379 ing, where the model learns to apply integration techniques (substitution, integration by parts) as
 380 intermediate steps rather than predicting the final answer directly.

381 Limitations

382 Our approach has several limitations. (1) Expression tree size is bounded (max depth 10, max
 383 512 tokens), excluding very large expressions encountered in some applications. (2) Verification
 384 depends on CAS differentiation plus numerical evaluation, which may return false negatives on
 385 complex special-function compositions where symbolic simplification fails to terminate within
 386 the timeout. (3) The tree GNN processes single trees rather than batched graphs via PyG-style
 387 `Batch.from_data_list`. This limits training throughput by $\sim 2\text{--}3\times$ compared to what batched
 388 graph processing would achieve (each forward pass handles one tree rather than a mini-batch of
 389 disjoint graphs packed into a single adjacency matrix). At inference time this is less impactful because
 390 sample-and-verify already generates N candidates sequentially per problem. Implementing batched
 391 graph encoding is straightforward—the scatter-based message passing already operates on node
 392 indices and would generalize to multi-graph batches with an offset—but we defer this optimization to
 393 future work as it does not affect model accuracy. (4) Our backward data generation cannot produce
 394 integration problems whose antiderivatives involve functions not in our expression grammar.

395 Broader Impact Statement

396 This work contributes to the automation of symbolic mathematics, a research area with both positive
 397 and cautionary implications.

398 **Positive impacts.** Automating symbolic integration can accelerate scientific computing in physics,
 399 engineering, and applied mathematics, where closed-form solutions are preferable to numerical
 400 approximations for efficiency, interpretability, and downstream algebraic manipulation. The parameter
 401 efficiency of our tree-native architecture (12.1M parameters vs. 95M for the seq2seq baseline of
 402 Lample and Charton [2020]) reduces the computational cost of training and inference, making neural
 403 integration more accessible to researchers without large GPU clusters.

404 **Potential risks.** We see no direct negative societal impacts from this work. The system produces
 405 mathematical expressions that are mechanically verifiable (differentiation), which limits the scope for
 406 harmful misuse. However, over-reliance on neural solvers without independent verification could
 407 propagate errors into downstream calculations. We mitigate this by emphasizing the verification
 408 oracle and by reporting failure rates alongside solve rates.

409 **Environmental considerations.** All experiments were conducted on a single GPU. Total compute
 410 for the paper (training, ablations, evaluation) is estimated at < 200 GPU-hours, a modest footprint
 411 compared to large language model training.

412 References

- 413 Guillaume Lample and François Charton. Deep learning for symbolic mathematics. In *ICLR*, 2020.
- 414 Robert H. Risch. The problem of integration in finite terms. *Transactions of the American Mathemat-*
 415 *ical Society*, 139:167–189, 1969.
- 416 Joel Moses. Symbolic integration: The stormy decade. *Communications of the ACM*, 14(8):548–560,
 417 1971.
- 418 Ernest Davis. The use of deep learning for symbolic integration: A review of (Lample and Charton,
 419 2019). *arXiv preprint arXiv:1912.05752*, 2019.

420 Miles Cranmer, Alvaro Sanchez-Gonzalez, Peter Battaglia, Rui Xu, Kyle Cranmer, David Spergel,
421 and Shirley Ho. Discovering symbolic models from deep learning with inductive biases. In
422 *NeurIPS*, 2020.

423 Luca Biggio, Tommaso Bendinelli, Alexander Neitz, Aurelien Lucchi, and Giambattista Parascandolo.
424 Neural symbolic regression that scales. In *ICML*, 2021.

425 Vighnesh Leonardo Shiv and Chris Quirk. Novel positional encodings to enable tree-based trans-
426 formers. In *NeurIPS*, 2019.

427 Mark Chen, Jerry Tworek, Heewoo Jun, et al. Evaluating large language models trained on code.
428 *arXiv preprint arXiv:2107.03374*, 2021.

429 Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural
430 message passing for quantum chemistry. In *ICML*, 2017.

431 Mert Ünsal, Timon Gehr, and Martin Vechev. AlphaIntegrator: Transformer action search for
432 symbolic integration proofs. *arXiv preprint arXiv:2410.02666*, 2024.

433 Vaibhav Sharma, Abhinav Nagpal, and Muhammed Fatih Balin. SIRD: Symbolic integration rules
434 dataset. In *MATH-AI Workshop at NeurIPS*, 2023.

435 Daniel Maître and Roi Santos-Mateos. Multi-variable integration with a neural network. *Journal of*
436 *High Energy Physics*, 2023(03):221, 2023.

437 Fizza Rubab, Ntumba Elie Nsambi, Martin Balint, Felix Mujkanovic, Hans-Peter Seidel, To-
438 bias Ritschel, and Thomas Leimkühler. Learning neural antiderivatives. *arXiv preprint*
439 *arXiv:2509.17755*, 2025.

440 Orestis Oikonomou, Levi Lingsch, Dana Grund, Siddhartha Mishra, and Georgios Kissas. Neuro-
441 symbolic AI for analytical solutions of differential equations. *arXiv preprint arXiv:2502.01476*,
442 2025.

443 Shu Wei, Yanjie Li, Lina Yu, Weijun Li, Min Wu, Linjun Sun, Jingyi Liu, Hong Qin, Yusong Deng,
444 Jufeng Han, and Yan Pang. Closed-form solutions: A new perspective on solving differential
445 equations. In *ICML*, 2025.

446 Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz
447 Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, 2017.

448 Qimai Li, Zhichao Han, and Xiao-Ming Wu. Deeper insights into graph convolutional networks for
449 semi-supervised learning. In *AAAI*, 2018.

450 Uri Alon and Eran Yahav. On the bottleneck of graph neural networks and its practical implications.
451 In *ICLR*, 2021.

452 Jake Topping, Francesco Di Giovanni, Benjamin Paul Chamberlain, Xiaowen Dong, and Michael M.
453 Bronstein. Understanding over-squashing and bottlenecks on graphs via curvature. In *ICLR*, 2022.

454 Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Łukasz Kaiser,
455 Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John
456 Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*,
457 2021.

458 Kazem Meidani, Parshin Shojaee, Chandan K. Reddy, and Amir Barati Farimani. SNIP: Bridging
459 mathematical symbolic and numeric realms with unified pre-training. In *ICLR (Spotlight)*, 2024.

460 Parshin Shojaee, Kazem Meidani, Amir Barati Farimani, and Chandan K. Reddy. Transformer-based
461 planning for symbolic regression. In *NeurIPS*, 2023.

462 Rashid Barket, Uzma Shafiq, Matthew England, and Juergen Gerhard. Transformers to predict the
463 applicability of symbolic integration routines. In *MATH-AI Workshop at NeurIPS*, 2024.

464 Rashid Barket, Matthew England, and Juergen Gerhard. Symbolic integration algorithm selection
465 with machine learning: LSTMs vs Tree LSTMs. In *ICMS*, 2024.

466 A Vocabulary

467 Table 4 lists the 256-token vocabulary used by the model.

Table 4: Token vocabulary (256 tokens).

Category	Tokens	Count
Special	PAD, SOS, EOS, UNK, [FEAT], [INDEF], [DEF], [VAR]	8
Binary ops	add, sub, mul, div, pow	5
Unary ops	neg	1
Elementary	sin, cos, tan, exp, log, sqrt, asin, acos, atan, sinh, cosh, tanh	12
Special fns	erf, Ei, Si, Ci, Li, BesselJ, BesselY, EllipticK, EllipticE, Gamma, digamma, FresnelS, FresnelC, Hyp2F1, polylog	15
Variables	x, y, z, w, t	5
Parameters	a...g	7
Constants	π , e , γ , Catalan’s constant	4
Numbers	sign marker (2) + base-100 digits (100)	102
Reserved	(unused, for future extensions)	97
Total		256

468 B Hyperparameters

Table 5: Full hyperparameter specification.

Category	Parameter	Value
Tree GNN encoder	Node dimension	256
	Message-passing rounds	8
	MLP update dims	$768 \rightarrow 512 \rightarrow 256$
	Variable-aware attn heads	8
	Dep-bias parameter β	learned scalar
Tree decoder	Cross-attention layers	8
	Attention heads	8
	FFN hidden dim (SwiGLU)	682
	Max decode depth	8
Training	Optimizer	AdamW
	Learning rate	3×10^{-4}
	LR warmup (linear)	5 epochs
	Schedule after warmup	Cosine annealing ($\eta_{\min} = 10^{-6}$)
	Weight decay	0.01
	Batch size	256
	Max epochs / patience	90 / 15
	Gradient clip norm	1.0
	Label smoothing	0.1
	Mixed precision	BFloat16 (AMP)
Inference	SWA start / lr	75% / 10^{-5}
	Temperature	0.7
	Top- p	0.95
	N (candidates)	25

469 C Data Generation Details

470 **Rust differentiation engine.** Our expression tree representation and symbolic differentiation
 471 are implemented in Rust with PyO3 bindings. The engine supports all operators and functions
 472 listed in Table 4, implements the chain rule for unary and binary compositions, and performs basic

473 algebraic simplification (identity absorption, constant folding, double negation elimination). Batch
 474 differentiation uses Rayon for parallel execution across CPU cores. Node identities are interned via
 475 `VarId` and `ParamId` handles, avoiding allocation on the hot path.

476 **Expression tree sampling.** Random expression trees are generated top-down with configurable
 477 parameters: maximum depth (1–10), maximum node count, variable set, parameter set, and operator
 478 weights. At each internal node, operator type is sampled with weights 40% binary, 25% unary
 479 elementary, 5% special function, and 30% leaf. This weighting produces a natural distribution of
 480 expression complexities.

481 **Skeleton extraction.** The structural skeleton of an expression is computed by replacing all leaf
 482 nodes (variables, constants, numbers) with a placeholder “_” while preserving the operator tree.
 483 For example, $\sin(x^2 + 3)$ and $\sin(y^2 + 7)$ share the skeleton `sin add pow _ _ _`. The skeleton
 484 determines the train/test split assignment.

485 D Verification Protocol

486 Our verifier accepts a candidate antiderivative $\hat{F}$ and a reference integrand f and returns **accept** iff
 487 $\frac{d}{dx}\hat{F}$ and f agree.

488 **Symbolic stage.** We first compute $\frac{d}{dx}\hat{F}$ symbolically in the Rust engine and compare against f
 489 modulo the basic algebraic simplifications implemented in the engine. If the normalized forms
 490 coincide, we accept.

491 **Numerical fallback.** If symbolic comparison is inconclusive, we evaluate both expressions at 20
 492 random points drawn uniformly from $[-5, 5]$ (avoiding poles) and accept iff at least 18/20 points
 493 match within relative tolerance 10^{-8} . For parametric tasks, parameters are instantiated with random
 494 concrete values at each evaluation point.

495 **False-positive analysis.** The numerical fallback introduces a nonzero false-positive probability:
 496 two distinct functions may agree at 18 or more of 20 random points. We bound this risk empirically by
 497 running the verifier on 10,000 known-incorrect (randomly paired) integrand–antiderivative pairs. The
 498 observed false-positive rate is $< 0.01\%$, consistent with the theoretical bound for random polynomial
 499 agreement. The symbolic stage (which resolves $> 97\%$ of cases) has zero false-positive risk by
 500 construction.

501 **False-negative analysis.** False negatives arise when the symbolic simplifier cannot reduce $\hat{F}' - f$ to
 502 zero and the numerical evaluation encounters cancellation or overflow. On special-function integrands
 503 with deeply nested compositions, we observe a $\sim 2\%$ false-negative rate (correct antiderivatives
 504 rejected due to numerical instability). These contribute to an underestimate of the true solve rate.

505 **Definite integrals.** For definite-integration tasks, we fall back to SymPy with `mpmath` quadrature,
 506 using a 4-second timeout. We acknowledge that this reintroduces the CAS-timeout limitation for this
 507 task type; the other four task types are handled entirely by the Rust path with no timeouts.

508 E NeurIPS Paper Checklist

- 509 1. **Claims.** Do the main claims made in the abstract and introduction accurately reflect the
 510 paper’s contributions and scope? **[Yes]** All claims are supported: architectural and dataset
 511 claims by construction; solve rate claims (Tables 2, 3) by experimental evaluation with
 512 3-seed averaging and error bars. Multivariate novelty claims are qualified as “symbolic
 513 antiderivatives for multivariate integrands” with acknowledgment that structurally this
 514 reduces to univariate integration with inert parameters.
- 515 2. **Limitations.** Does the paper discuss the limitations of the work performed by the authors?
 516 **[Yes]** A dedicated Limitations section discusses four limitations: bounded tree size, verifier
 517 false negatives, lack of graph batching, and grammar-bounded antiderivatives. The CE loss
 518 vs. equivalence class tension and sample correlation are discussed in Section 5.

3. **Theory.** If the work relies on theoretical results, does the paper include complete proofs? **[Yes]** The $1 - (1 - p)^N$ analysis (Section 3.6) is qualified as an upper bound under an independence assumption, with discussion of how inter-sample correlation degrades the bound. Complexity analysis (Section 3) derives GNN vs. Transformer costs from standard results.
4. **Experiments.** Does the paper include all information needed to understand and reproduce the experiments? **[Yes]**
 - Hyperparameters: Appendix B, matching code config.
 - Dataset: Section 3.2 and Appendix C.
 - Verification: Appendix D with false-positive/negative analysis.
 - Training: Section 4 (optimizer, schedule, curriculum, hardware).
 - Error bars: 3-seed mean $\pm$ std reported for all main results.
5. **Reproducibility.** Does the paper provide sufficient information for reproducing the experimental results? **[Yes]** Architecture, training config, data generation procedure, and evaluation protocol are fully specified. Code and data will be released upon acceptance.
6. **Code and data.** If the paper includes code or data, does the paper include all information needed to run the code and use the data? **[Yes]** Code release is planned upon acceptance, including the Rust data generation pipeline, Python training code, and evaluation scripts. A README with setup instructions will be provided.
7. **Training and test details.** Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, and what metrics are reported)? **[Yes]** Skeleton-based splits (Section 3.2), all hyperparameters (Appendix B), curriculum schedule (Section 4), and solve rate metric definition are specified.
8. **Error bars.** Does the paper report error bars (e.g., with respect to random seeds)? **[Yes]** All main results (Tables 2, 3) report mean $\pm$ std over 3 independent training runs with different random seeds.
9. **Compute.** Does the paper report the amount of compute required for each experiment? **[Yes]** Hardware (single A100 80GB), training time (~ 11 – 18 hours per run), and evaluation time (~ 30 – 60 minutes) are specified. Total compute estimated at < 200 GPU-hours.
10. **Code of ethics.** Does the research conducted in the paper conform to the NeurIPS Code of Ethics? **[Yes]** This work involves no human subjects, sensitive data, or dual-use concerns.
11. **Broader impacts.** Does the paper discuss both potential positive societal impacts and negative societal impacts of the work? **[Yes]** See Broader Impact Statement before the references section.
12. **Safeguards.** Does the paper describe safeguards that have been put in place for responsible release? **[N/A]** The system produces verifiable mathematical expressions with no direct safety concerns.
13. **Licenses.** Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected? **[Yes]** All code is original. PyTorch (BSD), SymPy (BSD), and Rust crates are properly licensed. No pre-existing datasets are used.
14. **New assets.** Are new assets introduced in the paper well documented and accompanied by a URL or other access information? **[Yes]** Dataset and code will be released upon acceptance with documentation.
15. **Crowdsourcing and research with human subjects.** Does the paper include the full text of instructions given to participants and details of compensation? **[N/A]** No crowdsourcing or human subjects.
16. **IRB approvals.** Does the paper describe potential risks for the participants and whether Institutional Review Board (IRB) approvals were obtained? **[N/A]** No human subjects.
17. **Declaration of LLM usage.** Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods? **[N/A]** Our core method does not use LLMs. The architecture is a task-specific GNN encoder–decoder trained from scratch on synthetic data. LLMs were used only as standard writing aids during manuscript preparation and are not part of the technical contribution.